

Produktrapport



Deltagere: Freja Pedersen og Kasper Korthsen

Projekttitel: WTLeaks

Startdato: 20. april, 2026

Slutdato: 20. maj, 2026

Skole: Mercantec, Viborg

Vejleder: Mathias Steenberg

Indholdsfortegnelse

Kravspecifikation.....	4
Projektbeskrivelse (Fælles).....	4
Formål (Fælles).....	4
Vejledning.....	5
Webapplikation (Freja).....	5
Login (Freja).....	5
App (Freja).....	5
Chat (Freja).....	5
Discovery (Freja).....	7
Indstillinger (Freja).....	8
Teknisk dokumentation.....	9
Backend (Kasper).....	9
Linux-server og Twingate (Freja).....	9
Node.js (Freja).....	10
Express.js (Freja).....	10
Socket.io (Freja).....	11
Filer og brug af bucket (Freja).....	13
PYTHON DELEN(Kasper).....	15
DATABASE.....	15
MAIN.....	15
MODELS DEL 1.....	17
MODELS DEL 2.....	19
SCHEMAS.....	19
EVENTS.....	20
SERVER.....	21
DEPS.....	22
USERS.....	22
MESSAGES DEL 1.....	24
MESSAGES DEL 2.....	24
AUTH DEL 1.....	25
AUTH DEL 2.....	26
AUTH DEL 3.....	26
AUTH DEL 4.....	27
AUTH DEL 5.....	29
AUTH DEL 6.....	30
ROOMS DEL 1.....	30
ROOMS DEL 2.....	31
ROOMS DEL 3.....	32
ROOMS DEL 4.....	34
ROOMS DEL 5.....	34

ROOMS DEL 6..... 35

Frontend (Freja).....40

 Login (Freja)..... 40

 App (Freja)..... 41

 Chat (Freja).....41

 Discovery (Freja).....46

 Indstillinger (Freja)..... 47

 Andet (Freja).....48

Kravspecifikation

Projektbeskrivelse (Fælles)

Vi vil udvikle en chat-applikation/platform, hvor brugere kan skrive med hinanden, enten i private eller offentlige rum. Brugere vælger selv, hvilke rum, de vil deltage i, og rummene identificeres med et ID og et navn. Offentlige rum kan brugere finde i en liste eller ved søgning, og private rum kan kun deltages i ved brug af givet id og adgangskode. Der er ikke nogen begrænsning for, hvor mange rum, en bruger kan deltage i. Beskeder er tilknyttet brugeren, der sender dem, og kan fjernes til hver en tid. Alle rum har en ejer og en valgfri mængde moderatorer, som kan fjerne både beskeder og brugere fra det tilknyttede rum. Brugere skal have fri adgang til at ændre bl.a. deres skærmenavn.

Formål (Fælles)

For vores brugere er målene med vores skalerbare chat-løsning, at de kan:

- Kommunikere i real-time
- Organisere samtaler i rum
- Dele filer
- Være sikre med deres data

For os er målene med dette projekt at vise vores forståelse og kompetencer indenfor:

- Real-time kommunikation
- Full Stack udvikling (frontend + backend)
- Håndtering af brugere
- Arbejde under tidspres

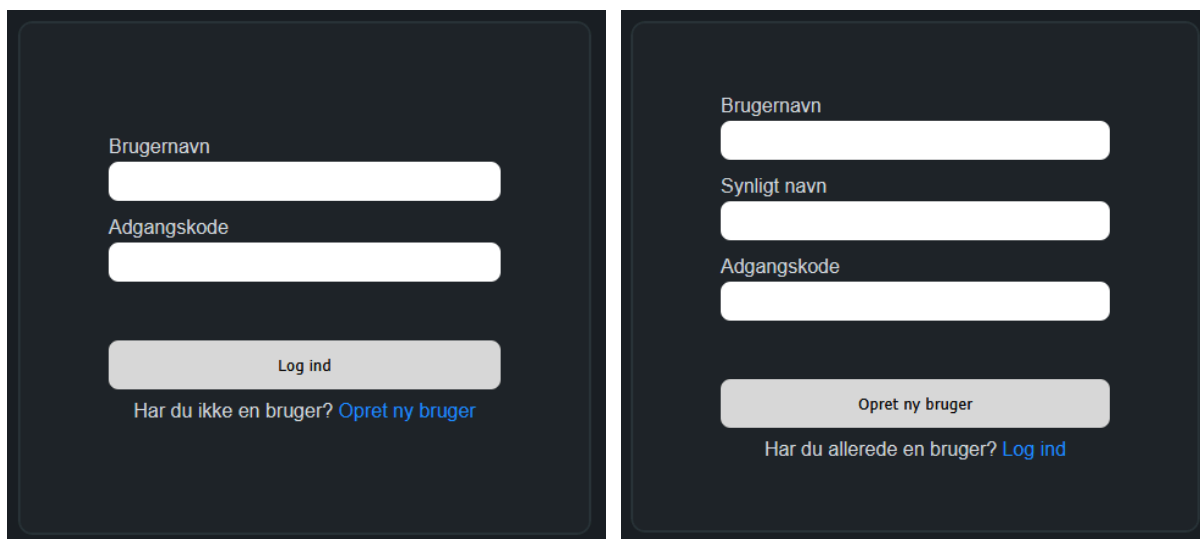
Vejledning

Webapplikation (Freja)

Denne del gennemgår webapplikationen, som kunden kan benytte som en del af vores platform. Der dækkes for begge websider, og alle de forskellige vinduer/menuer, som kunden kan støde på under brug.

Login (Freja)

For at kunne gøre brug af vores applikation, kræves det, at kunden har en bruger, som de kan logge ind med. Hvis kunden ikke har en bruger, kan de vælge at oprette en ved at trykke på den blå tekst i bunden af formularen. Brugernavnet skal være unikt. Hvis brugeren oprettes korrekt, giver browseren et lille pop-up, og formularen går tilbage til at spørge efter login-oplysninger.



The image shows two side-by-side screenshots of the Freja web application interface. The left screenshot shows the login form with fields for 'Brugernavn' (Username) and 'Adgangskode' (Password), a 'Log ind' button, and a link 'Har du ikke en bruger? Opret ny bruger' (Don't you have a user? Create new user). The right screenshot shows the registration form with fields for 'Brugernavn', 'Synligt navn' (Visible name), and 'Adgangskode', an 'Opret ny bruger' button, and a link 'Har du allerede en bruger? Log ind' (Do you already have a user? Log in).

Når kunden taster dem ind og logger ind som deres bruger, kan applikationen bruges.

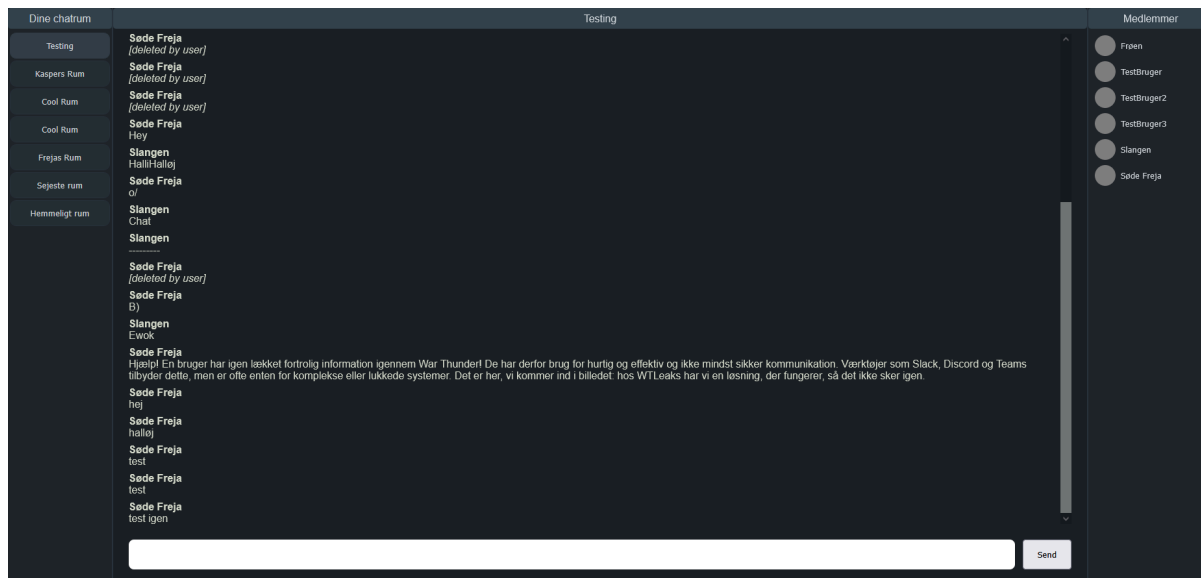
App (Freja)

I applikationen findes der 3 hovedmenyer: en til chat, en til at finde nye chatrum, og en til brugerens indstillinger. Disse tilgås henholdsvis via de store knapper i yderste venstre side.

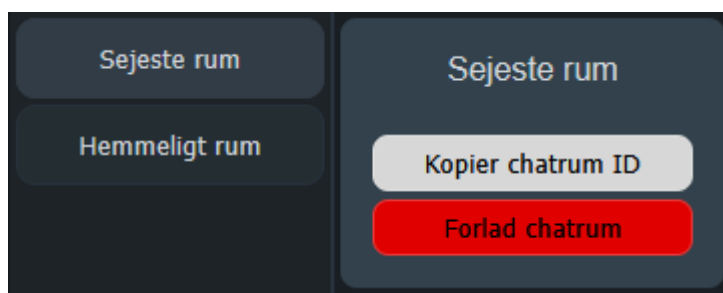
Chat (Freja)

Når brugeren trykker på chat-knappen, åbnes chat-siden. Her findes listen med de chatrum, brugeren er en del af. Hvis brugeren trykker på en knap tilhørende et chatrum, åbnes chatrummet og fylder resten af siden. Her ses både chat-loggen, der viser chatrummets beskedhistorik med tilhørende navne og indhold, samt de nuværende medlemmer af chatrummet.

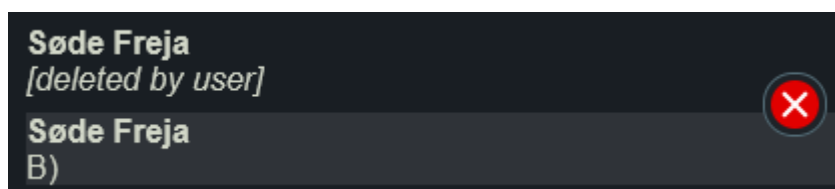




Hvis brugeren i stedet vælger at højreklikke på rummet, åbnes en lille kontekstmenu. Her kan brugeren vælge at kopiere et rums interne ID (i tilfælde af, at rummet er privat, er dette nyttigt hvis de gerne vil "invitere" andre med) eller forlade rummet. Brugeren kan ikke forlade sit eget rum, men får i stedet muligheden for at slette det samt al chat-historik. Denne menu kan lukkes ved at forsøge at åbne den igen, eller skifte til en anden side.

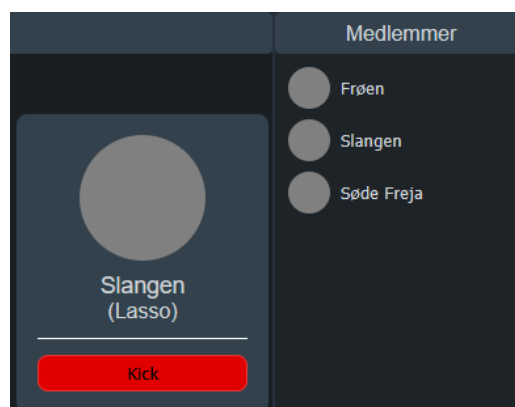


Brugeren kan sende beskeder i rummet ved at skrive i tekstboksen under chat-loggen, og så enten trykke på "Send"-knappen, eller ved brug af



"Enter"-tasten. Hvis brugeren fortryder en besked, de har sendt, kan de føre musen over sin besked, og så trykke på den røde X-knap, der findes mod højre. Hvis brugeren er administrator i rummet, kan de slette alle andres beskeder, og ikke kun sine egne.

Listen med rummets medlemmer kan ses yderst til højre. Hvis brugeren klikker på en anden bruger med enten venstre eller højre musetast, åbnes en lille biografi. Her kan den anden brugers synlige navn og brugernavn ses. Hvis brugeren er administrator i rummet, kan de sparke andre ud af rummet via en synlig "Kick"-knap, der så vil være til stede.



Discovery (Freja)

Her finder du chatrum, du kan deltage i. Du kan også oprette dine egne!

Opret nyt rum

Navn:

Adgangskode:

Privat rum? ☐

Opret nyt rum

Deltag i privat rum

Chat ID:

Adgangskode:

Deltag i rum

Discovery-siden bruges til at finde nye chatrum at deltage i. Øverst på siden findes to formularer; den venstre er til at oprette et nyt, offentligt chatrum, og den højre er til at deltage i et privat chatrum. Private chatrum bliver ikke vist i sektionen nedenunder, og kræver en adgangskode, hvorimod offentlige chatrum er åbne for alle.

Vis tilhørte rum? ☐

Filtrér rum: Brug

Kaspers Rum

Deltag

Cool Rum

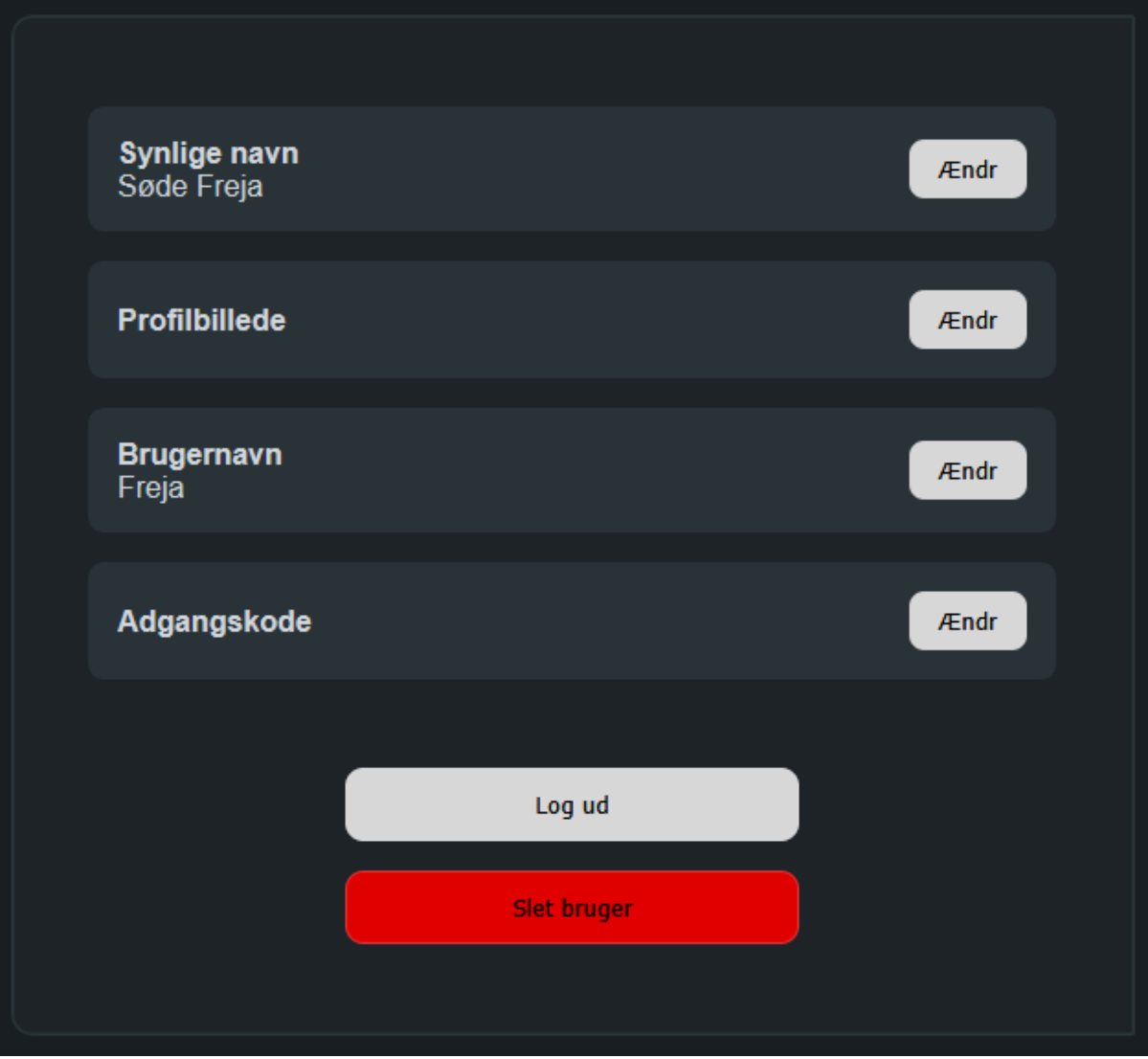
Deltag

Frejas Rum

Deltag

Under formularerne findes et gitter med offentlige chatrum. De er synlige med navn, og ønsker brugeren at deltage, skal de bare trykke på den tilhørende knap. Hvis det lykkedes at deltage, åbnes chat-menuen med rummet som det aktive chatrum.

Indstillinger (Freja)



Synlige navn Søde Freja	Ændr
Profilbillede	Ændr
Brugernavn Freja	Ændr
Adgangskode	Ændr

Log ud

Slet bruger

I denne menu kan brugeren ændre på sin konto. Her er der mulighed for at ændre på deres synlige navn, profilbillede, brugernavn eller adgangskode. Disse muligheder tilgås ved at trykke på deres tilknyttede "Ændr"-knap, hvorefter en forlængelse af elementet dukker op, og der er mulighed for at indtaste de nye, ønskede detaljer. Ændring af enten brugernavn eller adgangskode resulterer i, at brugeren bliver logget ud, og så skal logge ind igen med den nye kontodetalje. Profilbillede kan ændres ved at uploade et billede fra sin computer.

Nedenunder de forskellige muligheder for at ændre sin konto, findes der to knapper. Den øverste, "Log ud", logger brugeren ud. Dette invaliderer deres login, og fører dem til login-siden. Den nederste, "Slet bruger", sletter brugeren, hvorefter den ikke længere kan benyttes eller logges ind på.

Teknisk dokumentation

Backend (Kasper)

Backendten består af og er skrevet i Python dette er gjort da der er mest erfaring med dette og nogle af de forskellige frameworks er ikke så anderledes end hvad man måske er vant til, der er også blevet taget valget at bruge socket.io fremfor websocket, da det var nemmere at bruge i frontenden og er det der er mest erfaring med, dette bruger vi til at tilkoble os rummene, så er valget også faldet på FastApi da det fungerer basically ligesom Swagger UI og med routers som vi kender det, der er også taget valget at køre med Jwt tokens dette er gjort for at tilføje lidt ekstra sikkerhed, samt brugen af bcrypt til at hashe vores password og gjort brug af refresh tokens til at sikre os at dem der er logget ind ikke ryger af når deres access token stopper med at virke, herunder vil der være en gennemgang af strukturen i koden samt fremvisning af de forskellige dele af backendten samt server opsætningen, til vores backend del som har styring for frontenden og ikke forbindelse til databasen har vi gjort brug af server.js hvor vi har en som ligger udenfor som egentlig fungerer lidt ligesom main.py som sørger for at alt der har med node at gøre kører som vi gerne vil have det så har vi en src mappe hvor den anden server.js med alle socket handlers ligger og bliver sat sammen med python delens endpoints på FastAPIen så der er forbindelse gennem fra frontend til sockets og så til python backendten og derefter ender det i db.

Linux-server og Twingate (Freja)

Vores backend kører på en Ubuntu 24.04 Linux-server. Adgang til serveren er begrænset til medlemmer af vores private Twingate-netværk. Twingate er en service, der tilbyder Zero Trust Networking, og kan bruges til at finde adgang til resurser blandt specifikke brugere, selv indenfor et enkelt privat netværk, i modsætning til en mere traditionel VPN, hvor der er adgang til alt for alle medlemmer af et netværk. Twingate står for at godkende brugere og den nævnte begrænsning af adgang, så uvedkommende ikke kan oprette forbindelse til den. Forbindelser oprettes gennem en Connector, som vi har deployet på serveren via en Docker-container. De andre backend-dele er også opsat via Docker-containere, så de er samlede, og kan eventuelt skaleres ved behov.

Serveren er sat op med en bruger til hvert medlem, og begge er med i en gruppe kaldet "developers". Gruppen giver adgang til en fælles udviklermappe, `/home/dev`, hvor vores applikationer findes. Begge brugere har også sudo-adgang til at konfigurere Docker og andre opstillinger, der kræver det. Denne adgang er givet udelukkende fordi, vi kan stole på, at vi ikke ødelægger noget, og at vi undersøger tingene først (eller spørger), hvis vi er i tvivl om noget. Begge brugere har også SSH-nøgler til privat adgang fra vores primære arbejdsmaskiner. Da disse nøgler er gemt på selve maskinerne, er de meget svære at stjæle eller bruge til at undgå sikkerhedslag.

Idéen her er, at vi kan låse adgang til vores private resurser som database og API fuldstændigt af for omverdenen, men stadig have applikationen offentlig tilgængelig. Dette kan opnås ved brug af fx Cloudflare Tunnels eller en reverse proxy, men da vi ikke har erfaring med hverken af de to, er de, på grund af prioritering, ikke en del af dette projekt.

Node.js (Freja)

Som et bindeled mellem klienter og vores API, bruger vi en Node.js server. Vi bruger Node, da det er noget, vi er til dels bekendt med. Det er et rigtig udbredt værktøj, og har derfor også et kæmpe økosystem med pakker, der indeholder andre udvidelser og frameworks, vi kan gøre brug af. Dette gøres gennem Nodes egen package manager, npm. Selv om man skulle tro at npm står for det kreative navn "Node Package Manager", så er det faktisk "npm is not an acronym".

I vores projekt står Node for at styre kommunikationen mellem API og klient, og også for at sende diverse filer til klienten, som bl.a. de forskellige HTML-sider og deres tilknyttede scripts. Med Node kan vi også gøre brug af tidligere nævnte frameworks, som fx Express, men også andre dele middleware (dog kan der også ofte kobles middleware på de forskellige frameworks), som her primært bruges til at arbejde med cookies. Disse kan inkluderes i starten af vores server script, og dem, der kræver opsætning, kan sættes op længere nede efter at serveren defineres. Koden til vores Node-server findes i filen *server.js*.

```
import express from "express";
import { createServer } from "node:http";
import { Server } from "socket.io";
import { serialize, parse } from "cookie";
import cookieParser from "cookie-parser";
```

```
const app = express();
const httpServer = createServer(app);
const io = new Server(httpServer, {
  cors: {origin: "*" },
});

app.use(express.json());
app.use(cookieParser());
```

Express.js (Freja)

Vi gør brug af Express.js (eller bare Express) for at simplificere routing-processen. Frameworket gør det relativt nemt at dele de forskellige opgaver op i meget tydelige blokke, og så definere, hvad serveren skal gøre med hver af dem. Det er selvfølgelig muligt uden Express, men det gør det nemmere. Her er fx hvordan det ser ud, når en klient kobler sig på serveren:

Klienten får tilsendt *index.html*, som efterspørger både *app_client.js* og *style.css*, som browseren automatisk sender efterspørgsler på, og serveren så giver.

```
app.get("/", (req, res) => {
  res.sendFile(__dirname + "/index.html");
});

app.get("/app_client.js", (req, res) => {
  res.sendFile(__dirname + "/app_client.js");
});

app.get("/style.css", (req, res) => {
  res.sendFile(__dirname + "/style.css");
});
```

Andre steder, som i vores Node-specifikke endpoints til klienten, gøres der brug af lidt flere features i Express. Fordi kommunikation via WebSockets ikke er det samme som via HTTP, kommer der nogle småproblemer i at sende fx adgangsdetaljer. Den sikreste måde hvorpå vi kan gemme brugerens information hos brugeren selv, er via JSON Web Tokens, og specifikt ved at gemme dem som cookies med strenge regler. Det viser sig dog at være lidt af et problem at sende cookies frem og tilbage via websockets, som vi primært benytter til at kommunikere mellem Node-server og klient (og dækkes i næste del). For at løse det, bruges disse ekstra endpoints til klienten, hvor de kan sende en efterspørgsel til serveren, som så står for at sammensætte enten efterspørgsler til API'en, eller svar fra den, som klienten så får tilbage.

Hvis vi tager klient-login som et eksempel, så sender klienten sin information med brugernavn og adgangskode. Serveren tager disse, og videresender det til API'en. Hvis brugeren eksisterer, og detaljerne er korrekte, sender API'en tokens tilbage til serveren. Serveren tager så disse tokens, sætter dem ind som cookies i svaret til klienten, og sender dem. Dette sørger for, at vi kan bruge tokens som HttpOnly cookies, hvor scripts ikke kan nå dem.

```
app.post("/login/creds", async (req, res) => {
  const response = await OnClientTryLoginRequest(req);

  if (response.ok) {
    const json = await response.json();

    res.cookie("access_token", json.access_token, { httpOnly: true });
    res.cookie("refresh_token", json.refresh_token, { httpOnly: true });
    res.cookie("token_type", json.token_type, { httpOnly: true });

    res.status(response.status).send("Yay");
  }
  else {
    res.status(response.status).send("Nay");
  }
})
```

Socket.io (Freja)

Til selve chat-delen af vores chat-applikation, gør vi brug af Socket.io (som jeg bare kalder Socket fremover). Socket er et bibliotek, der primært bygger oven på WebSockets. WebSockets er en protokol, der gør brug af TCP sockets, og tillader kommunikation i begge retninger. Selv om protokollen er separat fra HTTP, etableres forbindelsen stadig med et HTTP handshake, og er bygget til at fungere sammen med HTTP-kommunikation. Kommunikation via WebSockets er event-drevet, og kan deles op lidt ligesom HTTP routing med forskellige navne på begivenheder.

```
socket.on("getOwnInfo", async (callback) => {
  callback(await OnSocketGetOwnInfo(socket));
});

socket.on("tryJoinRoom", (userID, roomID, password) => {
  OnSocketTryJoinChatroom(socket, userID, roomID, password);
});
```

På billedet her kan man se bl.a. "getOwnInfo" og "tryJoinRoom". Når klienten sender en begivenhed/et event med et navn som dem, så samler serveren den/det op, og gør så noget derfra. Disse kan sendes frit frem og tilbage, og der kan knyttes forskellig data på dem. De kan også sendes asynkront med en forventning om et svar, hvor modtageren sender et callback tilbage til sidst, som senderen kan læse.

```
const callback = await socket.emitWithAck("tryRegister", un, dn, pw);

if (callback.status !== "OK") {
  alert(callback.payload.message);
}
else {
  alert("Bruger oprettet!");
  SwapForms();
}
```

Siden at Socket også tillader en broadcast-model, kan man få individuelle klienter til at deltage i forskellige "rum", hvor intern kommunikation kan foregå. Det gør det relativt nemt at sørge for, at klienterne kan kommunikere med hinanden gennem serveren, uden at det når til uvedkommende.

Indtil videre kan brugere/sockets kun være medlem af ét rum ad gangen, men vi holder stadig øje med, hvilket rum de pt er i. Der kan eventuelt laves om på opstillingen, så activeRoomID bliver brugt i de forskellige broadcasts. Dvs at de kan være medlem af alle de forskellige Socket-rum, og kun modtage

```
if (socket.activeRoomID !== -1) {
  socket.leave(String(socket.activeRoomID));
}

socket.activeRoomID = roomID;
socket.join(String(roomID));

socket.emit("receiveChatroom", json);
```

beskederne i det aktuelle rum, og notifikationer om de andre (fx en rød prik). Dette har vi dog ikke gjort i denne udgave pga tidsbegrænsninger. Det kunne dog have gjort det nemmere at organisere nogle broadcasts, og burde måske overvejes til potentiel videreudvikling.

Der findes forskellige måder at sikre WS-forbindelser på, men grundet vores erfaring (eller mangel deraf), har vi primært gjort brug af JSON Web Tokens (JWT). JWT kan sagtens virke med WS, men vi er stødt ind i nogle problemer undervejs. For at gøre vores tokens mere sikre, har vi gemt dem som HttpOnly cookies hos brugeren. Det giver den fordel, at de ikke kan overskrives eller bruges af scripts,

og i stedet kun følger med HTTP-rutiner. WS laver et HTTP handshake når de forbinder, og kan udveksle cookies der. Det medfører dog nogle småproblemer - hvis vi vil have tokens med kort varighed, skal vi enten omtænke hvor de tokens gemmes på Node-serveren, eller aktivt sparke brugeren af, og så få dem til at forbinde igen til at lave et nyt handshake med de nye tokens, da cookies kun sendes med der. Grundet de problemer, og fordi vi allerede har et lag sikkerhed med vores access tokens, har vi valgt at droppe brugen af refresh tokens, da de ellers vil accelerere problematikken med handshakes. I et fremtidigt projekt ville det være oplagt at have det med til eftertanke, og eventuelt koble noget sikkerhed på selve WS-sessionen, da de er stateful og kan holde på information, der, gennem deres natur, ville være svær at stjæle.

Filer og brug af bucket (Freja)

I denne udgave af projektet nåede vi ikke at få gjort ordentlig brug af vores bucket. Opsætningen af den bliver dækket i et senere afsnit, og her bliver upload-processen kort gennemgået i stedet.

Socket kan bruges til at overføre filer. Den maksimale størrelse på filerne er som standard 1MB, men kan ændres via Node-serverens *maxHttpBufferSize* indstilling (udklip fra officiel Socket dokumentation til højre).

```
import { Server } from "socket.io";

const io = new Server({
  maxHttpBufferSize: 1e8 // 100 MB
});
```

Filer overført på denne måde bliver encoded til en (UTF-8) buffer, som er en type i Node. Disse buffere kan bruges til forskellige slags data til andre formål, men det bliver ikke dækket i detaljer her, fordi det bliver nemt meget teknisk og kommer uden for relevans.

Det endpoint vi bruger til at uploade billeder, forventer en fil encoded som formulardata (FormData), og en encoded buffer er hverken af disse ting. For at lave det om til en fil igen, sendes der også information om filtypen med som MIME-type, som bliver oversat til en filudvidelses-string.

Næste trin er at lave en ny fil ud fra vores buffer. En constructor til Nodes File-type tager en anden af Nodes typer, Blob. Blob-typen eksisterer som en indkapsling af rå data, og kan bygges med vores buffer. Så vi ender med at bygge en ny billedfil ved at bruge File-constructoren med vores nye Blob, et navn vi gerne vil have, filen skal have + filudvidelse, og så giver vi den MIME-typen som metadata for en god ordens skyld.

Når billedfilen er sammensat, laves et FormData-objekt, hvor kun filen er med. Dette objekt kan endelig sendes til vores API, som uploader billedet til vores bucket.

```
async function ChangeUserProfilePicture(socket, newPictureBuffer, fileType) {
  const cookies = parse(socket.handshake.headers.cookie);

  let fileTypeString;
  if (fileType === "image/png") {
    fileTypeString = ".png";
  }
  else if (fileType === "image/jpeg") {
    fileTypeString = ".jpg";
  }
  else {
    return;
  }

  const newFileName = socket.userID + "_pfp" + fileTypeString;

  const file = new File([
    new Blob([newPictureBuffer]),
    newFileName,
    {
      type: fileType
    }
  ]);

  const formData = new FormData();
  formData.append("file", file);

  const requestOptions = {
    method: "POST",
    headers: {
      "Authorization": "Bearer " + cookies.access_token
    },
    body: formData
  };
};
```

I øjeblikket er det ikke muligt faktisk at bruge billederne eller andre filer fra vores bucket - det har vi ikke nået. På samme måde har vi heller ikke nået fildeling i vores applikation, men vi er startet på processen her, og det kan eventuelt videreudvikles på for at blive en realiseret feature.

PYTHON DELEN(Kasper)

DATABASE.

Her har vi Database.py som egentlig i bund og grund kører det hele som laver den Base som vi basere backenden på,

Vi starter med at importere lidt forskelligt create_engine til at vi kører vores backend på,

sessionmaker til at kører sessionen,

og declarative_base så bruger vi OS og selvfølgelig den gode gamle dotenv til at hente vores env fil og benytte os af den,

det er også der koden starter med at hente vores env fil og så sætter vi databasen og på vores database url og derfra

starter vi vores sessionmaker som jo er local, autocommit er slået fra så der ikke sker ændringer som vi ikke vil have,

autoflush er også slået fra da det er en normal ting som SQL klarer for os, og derefter binder vi vores session til vores database og så kører det.

```
src > database.py > ...
1  from sqlalchemy import create_engine
2  from sqlalchemy.orm import sessionmaker, declarative_base
3  import os
4  from dotenv import load_dotenv
5
6  load_dotenv()
7
8  DATABASE_URL = os.getenv("DATABASE_URL")
9
10 engine = create_engine(DATABASE_URL)
11
12 Sessionlocal = sessionmaker(
13     autocommit=False,
14     autoflush=False,
15     bind=engine
16 )
17
18 Base = declarative_base()
```

MAIN.

Her har vi main.py som egentlig er vores kontrol center som sørger for at vores FastAPI er oppe og kører samt at vi har vores lifespan, CORS regler

og vores routes, og vores kombination af vores socket io og FastAPI og og så er der en print der er til for at tjekke at det er den korrekte URL,

Her starter vi egentlig selvfølgelig med at importere de forskellige ting som vi skal bruge til at køre det hele som f.eks vores modeller og API router, derefter starter vi med at kører vores async lifespan som er bundet op på vores create engine fra databasen så det køre samlet og så længe den er startet så kører den indtil den bliver slukket, så har vi CORS på der er ikke sat noget på iforhold til sikkerhed da det er internt og der ikke er nok viden omkring at have CORS kørende på en server og når det er et internt miljø er dette ikke tænkt som helt vildt vigtigt, derefter har vi vores routes som er defineret i hver deres fil og det kommer senere i rapporten men de er sat igang heri, derefter kombinere vi vores sockets og FastAPI så vi har en swaggerUI kørende som der kan testes igennem på de forskellige endpoints.


```
src > main.py > ...
1  from fastapi import FastAPI
2  from fastapi.middleware.cors import CORSMiddleware
3  from contextlib import asynccontextmanager
4
5  from src.database import engine
6  from src.models import Base
7
8  from api.routes import auth, rooms, messages, users
9  from sockets.server import sio
10
11
12  import socketio
13
14  # lifespan
15  @asynccontextmanager
16  async def lifespan(app: FastAPI):
17      Base.metadata.create_all(bind=engine)
18      yield
19
20  app = FastAPI(lifespan=lifespan)
21
22  app.add_middleware(
23      CORSMiddleware,
24      allow_origins=["*"],
25      allow_credentials=True,
26      allow_methods=["*"],
27      allow_headers=["*"],
28  )
29
30  # routes
31  app.include_router(auth.router)
32  app.include_router(rooms.router)
33  app.include_router(messages.router)
34  app.include_router(users.router)
35
36  # combine FastAPI and Socket.IO apps
37  app = socketio.ASGIApp(
38      sio,
39      other_asgi_app=app,
40  )
41
42  print(engine.url)
```

MODELS DEL 1.

Her er vores models.py som er jo egentlig er en meget god beskrivning af hvad det er vi starter selvfølgelig med at importere de forskellige ting vi skal

bruge, derefter starter vi med at etablere en relation til vores User model som er det de bliver tildelt når de joiner et rum derefter er der bare en "basic" model som bliver brugt samt det many-to-many relations som vi gør brug af, og derefter er der et moderator table som igen tager deres user_id og det rum de er i, derefter har vi så vores Room model som jo også har deres repræsentative modeller og relations og det er egentlig det og de bliver jo så hentet inde i f.eks main.py og andre steder vi gøre brug af modellerne.

```
src > models.py > ...
1  from sqlalchemy import Column, Integer, String, ForeignKey, Table, Boolean
2  from sqlalchemy.orm import relationship
3  from src.database import Base
4  from sqlalchemy import DateTime, Text
5  from datetime import datetime
6
7  # Association table for many-to-many relationship between users and rooms
8  user_room = Table(
9      "user_room",
10     Base.metadata,
11     Column("user_id", Integer, ForeignKey("users.id"), primary_key=True),
12     Column("room_id", Integer, ForeignKey("rooms.id"), primary_key=True)
13 )
14
15 # User model with relationships to rooms and messages
16 class User(Base):
17     __tablename__ = "users"
18
19     id = Column(Integer, primary_key=True, index=True)
20     username = Column(String, unique=True, nullable=False)
21     display_name = Column(String, nullable=True)
22     hashed_code = Column(String, nullable=False)
23     role = Column(String, nullable=False)
24
25     # Relationships
26     rooms = relationship("Room", secondary=user_room, back_populates="users")
27     messages = relationship("Message", back_populates="sender")
28
29 # Association table for many-to-many relationship between rooms and moderators
30 moderators_table = Table(
31     "room_moderators",
32     Base.metadata,
33     Column("user_id", Integer, ForeignKey("users.id"), primary_key=True),
34     Column("room_id", Integer, ForeignKey("rooms.id"), primary_key=True)
35 )
36
37 # Room model with relationships to users and messages
38 class Room(Base):
39     __tablename__ = "rooms"
40
41     id = Column(Integer, primary_key=True, index=True)
42     name = Column(String, nullable=False)
43     is_private = Column(Boolean, default=False)
44
45     owner_id = Column(Integer, ForeignKey("users.id"))
46
47     password_hash = Column(Integer, nullable=False)
48
49     # Relationships
50     owner = relationship("User")
51     users = relationship("User", secondary=user_room, back_populates="rooms")
52     moderators = relationship("User", secondary=moderators_table, back_populates="rooms")
53
54     # When a room is deleted, all associated messages will also be deleted due to the cascade option.
55     messages = relationship(
56         "Message",
57         back_populates="room",
58         cascade="all, delete-orphan"
59     )
60
```

MODELS DEL 2.

Her kan der ses resten af vores modeller og der er egentlig ikke så meget at sige om det for det er jo egentlig bare det samme som de tidligere viste modeller

hvor vi jo så bare her har de resternede modeller og de er jo så sat sammen med relations i Message modellen så det hele hænger sammen med user og room og

så vores RefreshToken og RoomRole så vi gør brug af.

```
59
60 # Message model with relationships to user and room
61 class Message(Base):
62     __tablename__ = "messages"
63
64     id = Column(Integer, primary_key=True, index=True)
65     content = Column(Text, nullable=False)
66     timestamp = Column(DateTime, default=datetime.utcnow)
67
68     # Status of the message, can be "ok", "deleted_user", or "deleted_admin"
69     status = Column(String, default="ok")
70     # "ok", "deleted_user", "deleted_admin"
71
72     sender_id = Column(Integer, ForeignKey("users.id", ondelete="SET NULL"), nullable=True)
73     room_id = Column(Integer, ForeignKey("rooms.id"))
74
75     # Relationships
76     sender = relationship("User", back_populates="messages")
77     room = relationship("Room", back_populates="messages")
78
79 class RefreshToken(Base):
80     __tablename__ = "refresh_tokens"
81
82     id = Column(Integer, primary_key=True)
83     token = Column(String, unique=True, nullable=False)
84
85     user_id = Column(Integer, ForeignKey("users.id"))
86     user = relationship("User")
87
88     expires_at = Column(DateTime, nullable=False)
89     revoked = Column(Boolean, default=False)
90
91 class RoomRole(Base):
92     __tablename__ = "room_roles"
93
94     id = Column(Integer, primary_key=True)
95     room_id = Column(Integer, ForeignKey("rooms.id", ondelete="CASCADE"))
96     user_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"))
97     role = Column(String, nullable=False)
98
```

SCHEMAS.

Her gør vi brug af et schema som egentlig er en dumbed down version af vores modeller og det gør at vi kun gør brug af de nødvendige ting såsom kode og username og gør det egentlig bare nemmere for os at håndtere forskellige ting at vi ikke skal have alt med, men kun det nødvendige.

```
src > schemas.py > ...
1  from pydantic import BaseModel
2
3  class UserCreate(BaseModel):
4      username: str
5      display_name: str = None
6      code: str
7
8  class UserLogin(BaseModel):
9      username: str
10     code: str
11
12  class RoomCreate(BaseModel):
13     name: str
14     is_private: bool = False
15     password: str | None = None
16
17  class MessageCreate(BaseModel):
18     room_id: int
19     content: str
20
21  class JoinRoomRequest(BaseModel):
22     password: str | None = None
23
24  class UpdateRequest(BaseModel):
25     username: str | None = None
26     display_name: str | None = None
27     current_password: str | None = None
28     new_password: str | None = None
29
30  class DisplayNameUpdate(BaseModel):
31     display_name: str
```

EVENTS.

Her er der integration af socketio på backend delen, mest af alt bruges det til at verificere at tokenen er med sant have styr på beskeder og så startes den sammen med [main.py](#).

```
sockets > events.py > connect
1  from sockets.server import sio
2  from core.jwt import verify_token
3
4
5  @sio.event
6  async def connect(sid, environ, auth=None):
7      token = auth.get("token") if auth else None
8
9      if not token:
10         return False
11
12         payload = verify_token(token)
13         if not payload:
14             return False
15
16         await sio.save_session(sid, {"username": payload["sub"]})
17
18
19  @sio.event
20  async def message(sid, data):
21      session = await sio.get_session(sid)
22
23      await sio.emit("message", {
24          "user": session["username"],
25          "message": data["message"]
26      })
27
28  def sio_app(app):
29      return sio.ASGIApp(sio, other_asgi_app=app)
```

SERVER.

Der er ikke rigtig så meget at sige her bare server delen som bruges andetsteds.

```
sockets > server.py > ...
1  import socketio
2
3  sio = socketio.AsyncServer(
4      async_mode="asgi",
5      cors_allowed_origins="*"
6  )
```

DEPS.

Det er hvor vi snupper vores sessionmaker som gør at vi kan starte vores server og her er logikken bagved til at starte den op

```
api > deps.py > ...
1  from src.database import Sessionlocal
2
3  def get_db():
4      db = Sessionlocal()
5      try:
6          yield db
7      finally:
8          db.close()
```

USERS.

Dette er vores router til at til det meste af vores User handling og der gør vi brug af igen nogle af de tidligere viste modeller og dependencies,

Vi starter jo selvfølgelig at definere vores router med hjælp af FastAPI, hvor vi har vores delete route som jo selvfølgelig sletter brugeren, og så

har vi også vores get profile hvor vi jo selvfølgelig henter det der er vigtig for brugeren at have adgang til på deres profil og så har vi en get på users udfra deres user id.

```
api > routes > users.py > get_user_by_id
1  from fastapi import APIRouter, Depends, HTTPException
2  from sqlalchemy.orm import Session
3
4  from src.models import User
5  from api.deps import get_db
6  from core.jwt import get_current_user
7
8  router = APIRouter()
9
10
11 @router.delete("/users/me")
12 def delete_user(
13     db: Session = Depends(get_db),
14     current_user: User = Depends(get_current_user)
15 ):
16     db.delete(current_user)
17     db.commit()
18
19     return {"status": "user deleted"}
20
21 @router.get("/me")
22 def get_my_profile(
23     current_user: User = Depends(get_current_user)
24 ):
25     return {
26         "id": current_user.id,
27         "username": current_user.username,
28         "display_name": current_user.display_name,
29         "rooms_id": [room.id for room in current_user.rooms]
30     }
31
32 @router.get("/users/{user_id}")
33 def get_user_by_id(
34     user_id: int,
35     db: Session = Depends(get_db),
36 ):
37     user = db.query(User).filter(User.id == user_id).first()
38
39     if not user:
40         raise HTTPException(404, "User not found")
41
42     return {
43         "username": user.username,
44         "display_name": user.display_name
45     }
```

MESSAGES DEL 1.

Her tager vi jo så og gør som de andre steder at vi henter de imports vi skal bruge samt at vi starter ud med at lave vores router, og så går vi jo videre til vores post som jo egentlig fungerer i tandem med nogle af de tidligere ting i modellerne, så den tjekker jo egentlig om rummet eksistere og der er jo dobbelt tjek på for vi tjekker også om de er medlem af rummet og så committer vi det til vores DB og det er egentlig det og så har vi en ligende function som i Users som er room id så vi jo selvfølgelig kan finde rummene.

```
api > routes > messages.py > delete_message
1  from fastapi import APIRouter, Depends, HTTPException
2  from sqlalchemy.orm import Session
3  from api.deps import get_db
4  from core.jwt import get_current_user
5  from core.permissions import ROLE_POWER, get_room_role
6  from src.models import Message, Room, User
7  from src.schemas import MessageCreate
8
9  router = APIRouter(prefix="/messages", tags=["messages"])
10
11 @router.post("/")
12 def create_message(msg: MessageCreate, db: Session = Depends(get_db), current_user: User = Depends(get_current_user)):
13
14     # Verify room exists
15     room = db.query(Room).filter(Room.id == msg.room_id).first()
16     if not room:
17         raise HTTPException(404, "Room not found")
18
19     # Verify user is a member of the room
20     if current_user not in room.users:
21         raise HTTPException(403, "Not a member of the room")
22
23     # Create and save the message
24     new_msg = Message(
25         content=msg.content,
26         room_id=msg.room_id,
27         sender_id=current_user.id
28     )
29
30     db.add(new_msg)
31     db.commit()
32     db.refresh(new_msg)
33
34     return new_msg
35
36 @router.get("/room/{room_id}")
37 def get_messages(room_id: int, db: Session = Depends(get_db)):
38     return db.query(Message).filter(Message.room_id == room_id).all()
39
```

MESSAGES DEL 2.

Her har vi vores delete route til messages som egentlig bare er en delete hvor vi bare har den del med at users kan slette deres egne messages og så har vi vores role power som egentlig gør at vi har rollerne Admin, Moderator og User, Admin og Moderator kan slette andre brugeres beskeder, det vises også ved sletning af beskederne som vises nederst med messages status og delete type


```
40
41 @router.delete("/{message_id}")
42 def delete_message(
43     message_id: int,
44     db: Session = Depends(get_db),
45     current_user: User = Depends(get_current_user)
46 ):
47     message = db.query(Message).filter(Message.id == message_id).first()
48
49     if not message:
50         raise HTTPException(404, "Message not found")
51
52     # Sender can always delete own message
53     if message.sender_id == current_user.id:
54         message.status = "deleted_user"
55         delete_type = "user"
56
57     else:
58         # Otherwise need moderator+
59         role = get_room_role(db, message.room_id, current_user.id)
60
61         if ROLE_POWER[role] < ROLE_POWER["moderator"]:
62             raise HTTPException(403, "Not allowed")
63
64         message.status = "deleted_admin"
65         delete_type = "admin"
66     db.commit()
67
68     return {"status": "message deleted", "type": delete_type}
```

AUTH DEL 1.

Her har vi starten på vores auth.py og der er selvfølgelig en meget større import af ting som vi skal bruge, samt har vi her gjort brug af minio til at kunne uploade profilbilleder til vores brugere, der er blevet oprettet en bucket gennem det som vi kan tilgå på web, og så har vi egentlig en ganske normal signup procedure hvor vi bare gør brug af hashing af koden og giver brugeren den standard rolle som user ved signup.

```
api > routes > auth.py > upload_profile_picture
1 from datetime import datetime
2
3 from fastapi import APIRouter, Depends, HTTPException, UploadFile, File
4 from sqlalchemy.orm import Session
5 from pydantic import BaseModel
6 from minio import Minio
7 from src.schemas import UpdateRequest, UserCreate, UserLogin, DisplayNameUpdate
8 from src.models import RefreshToken, User
9 from api.deps import get_db
10 from core.jwt import ALGORITHM, SECRET_KEY, create_refresh_token, get_current_user, hash_password, verify_password, create_access_token
11 from jose import jwt, JWTError
12
13 # This file contains the authentication routes for user signup, login, token refresh, and logout.
14 router = APIRouter(prefix="/auth", tags=["auth"])
15
16 from minio import Minio
17
18 client = Minio(
19     "minio:9000",
20     access_key="admin",
21     secret_key="supersuperpassword",
22     secure=False
23 )
24
25 # The signup route allows new users to create an account by providing a username and code. It checks if the username already exists and hashes the code before saving the user to the database.
26 @router.post("/signup")
27 def signup(user: UserCreate, db: Session = Depends(get_db)):
28     existing = db.query(User).filter(User.username == user.username).first()
29     # If a user with the same username already exists, it raises a 400 error. Otherwise, it creates a new user, hashes the code, and saves it to the database.
30     if existing:
31         raise HTTPException(400, "Username exists")
32     # The new user is created with the provided username and a hashed version of the code. The role is set to "user" by default. After saving the user, it returns a success message.
33     new_user = User(
34         username=user.username,
35         hashed_code=hash_password(user.code),
36         role="user"
37     )
38     # The new user is added to the database session, committed, and refreshed to get the new user's ID. Finally, a success message is returned.
39     db.add(new_user)
40     db.commit()
41     db.refresh(new_user)
42
43     return {"message": "User created"}
```

AUTH DEL 2.

Her er der endnu en hurtig en det er vores login logik hvor vi bare laver vores normale tjek om brugernavn og kode passer sammen og hvis de så bliver logget ind bliver de tildelt en access token og en refresh token og så er der et tjek på refresh token, som så tjekker om den er revoked eller ej, tokensne bliver så added in i vores DB til fremtidig tjeks ved login.

```
# The login route allows users to authenticate by providing their username and code. It verifies the credentials
@router.post("/login")
def login(data: UserLogin, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.username == data.username).first()
    # If the user is not found or the provided code does not match the stored hashed code, it raises a 401 error
    if not user or not verify_password(data.code, user.hashed_code):
        raise HTTPException(401, "Invalid credentials")
    # Creates the access and refresh token
    access_token = create_access_token(user)
    refresh_token, expires = create_refresh_token(user)

    db_token = RefreshToken(
        user_id=user.id,
        token = refresh_token,
        expires_at=expires,
        revoked=False
    )

    db.add(db_token)
    db.commit()
    db.refresh(db_token)

    return {"access_token": access_token, "refresh_token": refresh_token, "token_type": "bearer"}
```

AUTH DEL 3.

Her ser vi logikken bag vores refresh token, som først tjekker om vores refresh token er valid hvis ikke giver den selvfølgelig en error, og derefter

kommer logikken som udsteder en ny refresh token hvis altså access token er valid og det er så den der er valid til at sørge for at brugeren ikke logges ud ved at access token er expired.

```
69 # The refresh token route allows users to obtain a new access token by providing a valid refresh token. It verifies
70 @router.post("/refresh")
71 def refresh_token(refresh_token: str, db: Session = Depends(get_db)):
72     credentials_exception = HTTPException(
73         status_code=401,
74         detail="Invalid refresh token"
75     )
76     # The route first verifies the provided refresh token. If the token is invalid or expired, it raises a 401 e
77     try:
78         payload = jwt.decode(refresh_token, SECRET_KEY, algorithms=[ALGORITHM])
79
80         if payload.get("type") != "refresh":
81             raise credentials_exception
82
83         user_id = int(payload.get("sub"))
84
85
86     except (JWTError, ValueError):
87         raise credentials_exception
88     # The route first verifies the provided refresh token. If the token is invalid or expired, it raises a 401 e
89     db_token = db.query(RefreshToken).filter(
90         RefreshToken.token == refresh_token,
91         RefreshToken.revoked == False
92     ).first()
93
94     if not db_token:
95         raise credentials_exception
96     # If the token is valid, it retrieves the user associated with the token and generates a new access token an
97     if db_token.expires_at < datetime.utcnow():
98         raise credentials_exception
99
100     db_token.revoked = True
101
102     # If the token is valid, it retrieves the user associated with the token and generates a new access token an
103     user = db.query(User).filter(User.id == user_id).first()
104
105     new_access = create_access_token(user)
106     new_refresh, expires = create_refresh_token(user)
107
108     new_db_token = RefreshToken(
109         token=new_refresh,
110         user_id=user.id,
111         expires_at=expires,
112         revoked=False
113     )
114
115     db.add(new_db_token)
116     db.commit()
117
118     return {"access_token": new_access, "refresh_token": new_refresh, "token_type": "bearer"}
```

AUTH DEL 4.

Her har vi nogle forskellige logikker blandt andet vores logout, som egentlig bare logger folk ud og tager deres refresh token tilbage fordi at vi giver dem en ny en ved login, og så har vi delete user som jo er at slette brugeren fra vores database af, derefter har vi en get på useren til at indhente dataen om brugeren, samt har vi en hvor vi bruger user ud fra id så vi kan hvilke rum de er med i.

```
@router.post("/logout")
def logout(refresh_token: str, db: Session = Depends(get_db)):

    db_token = db.query(RefreshToken).filter(
        RefreshToken.token == refresh_token).first()

    if db_token:
        db_token.revoked = True
        db.commit()

    return {"message": "Logged out"}

@router.delete("/users/me")
def delete_user(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user)
):
    # create a unique anonymized username so unique constraints stay satisfied
    anon = f"deleted_user_{current_user.id}_{int(datetime.utcnow().timestamp())}"
    current_user.username = anon
    current_user.display_name = "Deleted user"
    current_user.hashed_code = "" # clear authentication secret

    # delete refresh tokens so the user can be anonymized safely
    db.query(RefreshToken).filter(RefreshToken.user_id == current_user.id).delete()

    db.commit()
    return {"status": "user anonymized"}

@router.get("/me")
def get_my_profile(
    current_user: User = Depends(get_current_user)
):
    return {
        "id": current_user.id,
        "username": current_user.username,
        "display_name": current_user.display_name,
        "rooms_id": [room.id for room in current_user.rooms]
    }

@router.get("/users/{user_id}")
def get_user_by_id(
    user_id: int,
    db: Session = Depends(get_db),
):
    user = db.query(User).filter(User.id == user_id).first()

    if not user:
        raise HTTPException(404, "User not found")

    return {
        "username": user.username,
        "display_name": user.display_name,
        "rooms_id": [room.id for room in user.rooms]
    }
```

AUTH DEL 5.

Her er der en lidt længere logik til at opdatere brugeren fordi at vi bliver nød til at tænke det at lave en ændring som et ny login og det betyder nye tokens vi tjekker selvfølgelig først om brugeren eksistere og så skal vi selvfølgelig sørge for at de klan deres kode før at vi giver dem mulighed for at ændre den så det er ret basic logik, men bare med lidt mere på for at sørge for at der ikke sker fejl med login grundet tokens.

```
170 @router.patch("/update")
171 def update_me(
172     data: UpdateRequest,
173     db: Session = Depends(get_db),
174     current_user: User = Depends(get_current_user)
175 ):
176     if data.username is not None:
177         existing = db.query(User).filter(
178             User.username == data.username,
179             User.id != current_user.id
180         ).first()
181
182         if existing:
183             raise HTTPException(400, "Username exists")
184
185         current_user.username = data.username
186
187     new_access = None
188     new_refresh = None
189
190     if data.new_password is not None:
191         if not data.current_password:
192             raise HTTPException(400, "Current password required")
193
194         if not verify_password(
195             data.current_password,
196             current_user.hashed_code
197         ):
198             raise HTTPException(400, "Current password incorrect")
199
200         current_user.hashed_code = hash_password(data.new_password)
201
202     db.query(RefreshToken).filter(
203         RefreshToken.user_id == current_user.id,
204         RefreshToken.revoked == False
205     ).update({
206         "revoked": True
207     })
208
209     new_access = create_access_token(current_user)
210     new_refresh, expires = create_refresh_token(current_user)
211
212     db.add(
213         RefreshToken(
214             token=new_refresh,
215             user_id=current_user.id,
216             expires_at=expires,
217             revoked=False
218         )
219     )
220
221     db.commit()
222
223     response = {
224         "status": "profile updated",
225     }
226
227     if new_access:
228         response["access_token"] = new_access
229         response["refresh_token"] = new_refresh
230
231     return response
232
```

AUTH DEL 6.

Det her er bare vores basic update til display navn og siden det ikke ændre noget drastisk som ved det andet behøves tokens ikke at ændres i denne sammenhæng, derefter har vi vores post til at kunne uploade profilbilleder til vores bucket og det sker som man typisk gør her har vi bare navn på billedet og hvilken type det er dette er gjort muligt med noget der kaldes multipart form, hvilket gør det muligt at modtage alle data former som f.eks Jpeg og PNG filer samtidig, uden at klienten der uploader skal til at ændre deres billede format.

```
@router.patch("/updateDisplay")
def update_display_name(
    data: DisplayNameUpdate,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user)
):
    current_user.display_name = data.display_name
    db.commit()
    return {"status": "display name updated"}

@router.post("/upload-profile-picture")
async def upload_profile_picture(
    file: UploadFile = File(...)
):
    object_name = file.filename

    contents = await file.read()

    from io import BytesIO

    client.put_object(
        "profil",
        object_name,
        BytesIO(contents),
        length=len(contents),
        content_type=file.content_type
    )

    return {
        "status": "uploaded",
        "file": object_name
    }
```

ROOMS DEL 1.

Her starter vi som vi altid gør henter de nødvendige imports der skal bruges og derefter definere vi vores route, så har vi en basic post til at oprette rooms,

dette gøres ved at du skal have navn og så bliver ejerens id knyttet til det da det bruges til at tildele admin rolen over rummet, vi har også gjort det muligt at lave et privat rum og der tænkte vi at man skal have kode på da det giver mest mening denne kode bliver også hashed.

```
api > routes > rooms.py > get_rooms
1  from fastapi import APIRouter, Depends, HTTPException
2  from sqlalchemy.orm import Session
3  from sqlalchemy import text, exists
4  from api.deps import get_db
5  from core.jwt import get_current_user, hash_password, verify_password
6  from core.permissions import require_role
7  from src.schemas import RoomCreate, JoinRoomRequest
8  from src.models import Room, User, user_room
9
10 router = APIRouter(prefix="/rooms", tags=["rooms"])
11
12
13 @router.post("/")
14 def create_room(
15     room: RoomCreate,
16     db: Session = Depends(get_db),
17     current_user: User = Depends(get_current_user)
18 ):
19     new_room = Room(
20         name=room.name,
21         is_private=True if room.password else room.is_private,
22         owner_id=current_user.id,
23         password_hash=hash_password(room.password) if room.password else None
24     )
25
26     db.add(new_room)
27     db.commit()
28     db.refresh(new_room)
29
30     return {
31         "id": new_room.id,
32         "name": new_room.name,
33         "is_private": new_room.is_private
34     }
35
```

ROOMS DEL 2.

Her er logikken bag at kunne joine et rum, dette havde vi lidt af et problem med grundet sqlalchemy queries, men det er blevet løst ved at køre querien i koden istedet for at have sqlalchemy til at gøre det, vi tjekker selvfølgelig om rummet eksistere og tjekker om det er privat og der vil der jo så blive tjekket kode og så tager vi jo så og tilføjer rummet til brugeren og det vil blive vist i deres liste over rum de er med i.

```
@router.post("/{room_id}/join")
def join_room(
    room_id: int,
    data: JoinRoomRequest,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user)
):
    room = db.query(Room).filter(Room.id == room_id).first()

    if not room:
        raise HTTPException(404, "Room not found")

    if room.password_hash:
        if not data.password or not verify_password(data.password, room.password_hash):
            raise HTTPException(403, "Invalid room password")

    existing = db.execute(
        text("""
            SELECT 1
            FROM user_room
            WHERE user_id = :user_id
            AND room_id = :room_id
        """),
        {
            "user_id": current_user.id,
            "room_id": room.id
        }
    ).fetchone()

    if existing:
        return {"status": "already joined"}

    db.execute(
        text("""
            INSERT INTO user_room (user_id, room_id)
            VALUES (:user_id, :room_id)
        """),
        {
            "user_id": current_user.id,
            "room_id": room.id
        }
    )

    db.commit()

    return {"status": "joined"}
```

ROOMS DEL 3.

Her har vi logikken bag både at slette et rum som kun ejeren kan gøre og der bliver der selvfølgelig tjekket om det er ejeren før det slettes, og derefter

har vi vores logik omkring at kicke en user fra rummet, vi tjekker om brugeren er moderator eller over før vi kicker nogen fra rummet og tjekker selvfølgelig

også om brugeren overhovedet er i rummet, men dette er ikke et problem derefter bliver rummet selvfølgelig slettet fra deres oversigt gennem DB.

```
@router.delete("/{room_id}")
def delete_room(
    room_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user)
):
    room = db.query(Room).filter(Room.id == room_id).first()

    if not room:
        raise HTTPException(404, "Room not found")

    require_role(db, room_id, current_user.id, "owner")

    db.delete(room)
    db.commit()

    return {"status": "room deleted"}

@router.delete("/{room_id}/users/{user_id}")
def kick_user(
    room_id: int,
    user_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user)
):
    room = db.query(Room).filter(Room.id == room_id).first()
    if not room:
        raise HTTPException(404, "Room not found")

    require_role(db, room_id, current_user.id, "moderator")

    # 🔍 Check membership directly in DB
    result = db.execute(
        text("SELECT 1 FROM user_room WHERE user_id = :u AND room_id = :r"),
        {"u": user_id, "r": room_id}
    ).first()

    if not result:
        raise HTTPException(404, "User is not in the room")

    # ✅ Delete membership safely
    db.execute(
        text("DELETE FROM user_room WHERE user_id = :u AND room_id = :r"),
        {"u": user_id, "r": room_id}
    )

    db.commit()

    return {"status": "user kicked"}
```

ROOMS DEL 4.

Her har vi logikken bag at få vores rooms ud til brugeren, så vi tjekker deres id og tjekker hvilke rooms de er med i som de så får som en liste.

```
@router.get("/my")
def get_my_rooms(
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user)
):
    rooms = current_user.rooms

    return [
        {
            "id": room.id,
            "name": room.name,
            "members": [
                {
                    "id": user.id,
                    "display_name": user.display_name
                }
                for user in room.users
            ]
        }
        for room in rooms
    ]
```

ROOMS DEL 5.

Her har vi logikken bag at få vores users og messages ud på vores room, så vi har styr på hvem er ejeren hvem er medlemmer og vi har en måde at vise hvem der har sendt hvad samtidig med at vi kan se hvem der har slettet beskederne i rummet.

```
@router.get("/{room_id}")
def get_room_details(
    room_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user)
):
    room = db.query(Room).filter(Room.id == room_id).first()

    if not room:
        raise HTTPException(404, "Room not found")

    if current_user not in room.users:
        raise HTTPException(403, "Not member of room")

    return {
        "id": room.id,
        "name": room.name,
        "owner_id": room.owner_id,

        "members": [
            {
                "id": user.id,
                "username": user.username,
                "display_name": user.display_name
            }
            for user in room.users
        ],

        "messages": [
            {
                "id": msg.id,
                "content": ( "[deleted by user]" if msg.status == "deleted_user"
                             else "[deleted by admin]" if msg.status == "deleted_admin"
                             else msg.content
                ),
                "status": msg.status,
                "sender_id": msg.sender_id
            }
            for msg in room.messages
        ]
    }
```

ROOMS DEL 6.

Her har vi vores leave som gør at brugere kan forlade rummet de er medlem af og det er bare ret basic, der er så sat en begrænsning på at owner ikke kan leave rummet da dette ville skabe problemer og det giver mest mening at slette rummet hvis man alligevel forlader det, og så har vi vores liste hvor vi tjekker om rummet er privat for at det ikke bliver vist på listen over rum.

```
@router.post("/{room_id}/leave")
def leave_room(
    room_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user)
):
    room = db.query(Room).filter(Room.id == room_id).first()

    if not room:
        raise HTTPException(404, "Room not found")

    # Check membership using explicit query against the association table
    member = db.execute(
        text("SELECT 1 FROM user_room WHERE user_id = :u AND room_id = :r"),
        {"u": current_user.id, "r": room_id}
    ).first()

    if not member:
        raise HTTPException(400, "You are not in this room")

    # Optional: prevent owner from leaving
    if room.owner_id == current_user.id:
        raise HTTPException(400, "Owner cannot leave their own room")

    # Perform a safe delete on the association row
    result = db.execute(
        text("DELETE FROM user_room WHERE user_id = :u AND room_id = :r"),
        {"u": current_user.id, "r": room_id}
    )

    # Log if nothing was deleted (can indicate concurrent modification)
    try:
        rowcount = result.rowcount
    except Exception:
        rowcount = None

    if rowcount == 0:
        logger.warning("leave_room: expected to remove 1 membership row but removed 0 (user=%s room=%s)", current_user.id, room_id)

    db.commit()

    return {"status": "left room"}

@router.get("/")
def get_rooms(
    db: Session = Depends(get_db)
):
    rooms = db.query(Room).filter(Room.is_private == False).all()

    return [
        {
            "id": room.id,
            "name": room.name,
            "owner_id": room.owner_id
        }
        for room in rooms
    ]
```

DOCKER-COMPSE.yml

Her er vores docker compose fil som egentlig har alle vores services inde i til at starte når det er vi starter vores docker

op, og det er hele køre jo ud fra den ene så er der hvad version vi vil køre og hvilke filer de har som env. og requirements.text

men req er kun til python fungere som npm install i node

```

dev > docker-compose.yml
1  services:
2
3    api:
4
5      container_name: fastapi_api
6
7      ports:
8        - "8000:8000"
9
10     depends_on:
11       - db
12       - minio
13
14     env_file:
15       - .env
16
17     restart: unless-stopped
18
19
20   minio:
21     image: minio/minio:latest
22     container_name: minio_bucket
23
24     ports:
25       - "9000:9000"
26       - "9001:9001"
27
28     environment:
29       MINIO_ROOT_USER: admin
30       MINIO_ROOT_PASSWORD: supersecretpassword
31
32     volumes:
33       - minio_data:/data
34
35     command: server /data --console-address ":9001"
36
37     restart: unless-stopped
38
39   node_app:
40     build:
41       context: .
42       dockerfile: Dockerfile.node
43
44     ports:
45       - "3000:3000"
46
47     env_file:
48       - .env
49
50   db:
51     image: postgres:16
52     container_name: postgres_db
53
54     restart: always
55
56     environment:
57       POSTGRES_USER: Kasper
58       POSTGRES_PASSWORD: H5kode
59       POSTGRES_DB: PostDB
60
61     ports:
62       - "5432:5432"
63
64     volumes:
65       - postgres_data:/var/lib/postgresql/data
66
67
68   volumes:
69     postgres_data:
70     minio_data:

```

REQ

Her har vi requirements.txt der er ikke så meget at sige det er bare de libraries vi gør brug af som bliver hentet ved start af serveren

```
dev > requirements.txt
1 fastapi
2 uvicorn[standard]
3 sqlalchemy
4 psycopg2-binary
5 passlib[bcrypt]
6 python-jose
7 python-dotenv
8 python-socketio
9 bcrypt==4.0.1
10 minio
11 python-multipart
```

DOCKER PYTHON

Her er vores docker fil til python med alt vi skal bruge til at starte tager den fra req.txt og så installere før den køre alt det andet igennem som uvicorn osv.

```
dev > Dockerfile
1 FROM python:3.11
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6
7 RUN pip install --no-cache-dir -r requirements.txt
8
9 COPY . .
10
11 WORKDIR /app/h5-msp-wtleaks
12
13 CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8000"]
14
```

DOCKER NODE

Node docker filen fungerer egentlig på samme måde som python dockeren bare andre kommandoer.

```
dev > Dockerfile.node
1 FROM node:20-alpine
2
3 WORKDIR /app
4
5 COPY h5-msp-wtleaks/src/node/package*.json .
6
7 RUN npm install
8
9 COPY h5-msp-wtleaks/src/node/ .
10
11 EXPOSE 3000
12
13 # Start the Node.js app
14 CMD ["node", "server.js"]
15
```


Frontend (Freja)

Vores frontend består af en blanding af HTML, CSS, JavaScript og Socket. Da Socket allerede er dækket i Backend-delen, bliver det ikke forklaret her. I stedet gennemgår dette afsnit nogle mere centrale processer fra brugerens perspektiv. De relevante filer til frontendens kode er *app_client.js* og *login_client.js*.

Login (Freja)

Login-sidens HTML-struktur har to næsten identiske dele. De er begge placeret i en div midt på siden, og når man trykker på knappen til at skifte formularer, bytter de to dele synlighed. For at få det til at fungere, starter registreringsformularen med at være usynlig (*display: none* via "Hidden"-klassen).

```
<div class="ContentDiv">
  <div id="loginDiv" class="CentreDiv LightText">
    <div id="loginForm" class="LoginContainer">
      <div class="FormPair">
        <label for="loginUsername">Brugernavn</label>
        <input id="loginUsername" type="text">
      </div>
      <div class="FormPair">
        <label for="loginPassword">Adgangskode</label>
        <input id="loginPassword" type="password">
      </div>
      <button class="FormButton" onclick="TryLogInRequest()">Log ind</button>
      <div>
        <span>Har du ikke en bruger? </span>
        <span class="RegisterSpan" onclick="SwapForms()">Opret ny bruger</span>
      </div>
    </div>
  </div>
```

```
// Skifter mellem login og registrering
function SwapForms() {
  if (!Number(formState)) {
    loginForm.classList.add("Hidden");
    registerForm.classList.remove("Hidden");
  }
  else {
    registerForm.classList.add("Hidden");
    loginForm.classList.remove("Hidden");
  }

  formState = 1 - Number(formState);
}
```

På login-siden findes der et større brug af HTTP Requests. Da vi gerne vil have, at brugeren skal holde på nogle tokens, og de ikke skal røres af scripts, kan vi ikke gennemføre efterspørgsler, der involverer tokens, gennem sockets.

Når brugeren logger ind, får de tilsendt tokens som svar. For at få vores socket-forbindelse til at stemme overens med de nye tokens (og de oplysninger, de indeholder), bliver den genstartet lige bagefter.

```
try {
  const response = await fetch("/login/creds", requestOptions);

  if (response.ok) {
    socket.disconnect().connect();
    // Åbner siden til applikationen, hvis man kan logge ind
    window.open("/", "_self");
  }
}
```

Dette betyder, at serveren og klienten laver et nyt handshake, og der forhåbentligt ikke hænger gamle oplysninger på klienten, da det ellers kan skabe en del problemer i vores opstilling.

App (Freja)

Vores applikation er betydeligt mere involveret og kompliceret. Til forskel fra login-siden, gemmes der en del information lokalt her, bl.a. for at spare på API-kald, men også for at gøre det nemmere at tilgå relevant (og ikke privat) data. Variablerne bliver forklaret i løbet af dette afsnit.

Når brugeren logger ind, sendes der automatisk en efterspørgsel på information om brugeren selv, så det kan gemmes i *thisUser*. Derudover modtages der også information om hvilke rum, brugeren er en del af, som så gemmes i *cachedRoomIDs*.

```
let activeChatroomContextMenuID = -1;
let activeMiniBioID = -1;

let thisUser = null;

let knownExternalUsers = [];
let cachedRoomIDs = [];

let currentTab = "";
let currentRoomID = -1;
let currentRoom = null;

let discoveryRoomsIDs = [];
let discoveryFilterMember = true;
```

```
async function GetOwnInfo() {
  const callback = await socket.emitWithAck("getOwnInfo");

  if (callback.status === "OK") {
    thisUser = new User(callback.payload.id, callback.payload.username, callback.payload.display_name);
    cachedRoomIDs = callback.payload.rooms_id;

    const settingsDisplayName = document.getElementById("settingsDisplayName");
    settingsDisplayName.innerText = thisUser.DisplayName;

    const settingsUserName = document.getElementById("settingsUserName");
    settingsUserName.innerText = thisUser.UserName;
  }
}
```

Chat (Freja)

Når chat-menuen åbnes, laves der et fetch på de rum, brugeren er medlem af. Her modtages der information som deres ID, navn og ejer-ID. Dette bruges til at lave en liste på siden, hvor de opstilles med navn. Hvis man højreklikker på dem, kommer der en overskrevet kontekstmenu, hvor man kan kopiere rummets ID eller enten forlade eller slette rummet, afhængigt af privilegier. Åbnes denne

menu, sættes *activeChatroomContextMenuID* til det rums ID, og lukkes det igen, sættes variabelen til -1 (dvs intet nuværende rum). Dette bruges til at holde styr på, hvornår klik skal åbne, lukke, eller opdatere den lille menu.

```
// Listen af chatrum på siden
function OnReceiveChatroomList(roomList) {
    const chatList = document.getElementById("chatList");

    for (let i = 0; i < roomList.length; i++) {
        const room = roomList[i];

        const roomListing = document.createElement("div");
        roomListing.classList.add("ChatListing");

        const button = document.createElement("button");
        button.classList.add("LightText");
        button.id = "crlid" + room.id;
        button.innerText = room.name;

        button.onclick = function () { socket.emit("tryEnterRoom", room.id) };

        button.addEventListener("contextmenu", function(event) {
            event.preventDefault();
            ToggleChatroomContextMenu(room.name, room.id, room.owner_id);
        }, false);

        roomListing.appendChild(button);

        chatList.appendChild(roomListing);
    }

    SetHighlightForCurrentChatroom(true);
}
```

Hvis brugeren trykker på et af disse chatrum, laves der endnu et fetch, men på det specifikke chatrum, og med mange flere detaljer - her gives også en detaljeret medlemsliste og en chat-log. Menuen bliver mere eller mindre nulstillet, så gamle detaljer ikke skaber konflikt, og både *currentRoom* og *currentRoomID* sættes til fremtidig reference fra andre steder i koden.

Bagefter bygges selve chat-loggen ud fra den information, der modtages fra serveren. Selve chatLog-elementet udskiftes og sættes ind i DOMen til sidst for at spare på DOM-opdateringer, da ændringer af elementer allerede i DOMen opdaterer det hele, og der er utroligt mange tilføjelser for at bygge alle beskederne korrekt (proces ikke vist her).

```
async function OnReceiveChatroom(room) {
  SetHighlightForCurrentChatroom(false);
  ResetMiniBio();

  currentRoomID = room.id;
  currentRoom = room;

  const messages = room.messages;

  if (chatLogContainer.children.length !== 0) {
    chatLogContainer.removeChild(chatLogContainer.firstChild);
  }

  DoShowChatFeatures(true);
  SetHighlightForCurrentChatroom(true);

  chatWindowHeaderSpan.innerText = room.name;

  chatLog = document.createElement("div");
  chatLog.classList.add("ChatLog");
}
```

Da der kun modtages information om selve rummet, kan man løbe ind i et meget specifikt problem: Hvad nu hvis der er beskeder fra tidligere medlemmer? De findes jo ikke på listen! Som løsning kan der laves et fetch på et bestemt bruger-ID, som giver den samme information, som vores fetch på chatrummet gjorde. For at spare på API-kald, gemmes information om denne slags brugere i en separat liste, *knownExternalUsers*, som applikationen tjekker allerførst, hvis brugeren ikke er medlem af rummet. Den eneste ulempe her er, at hvis den bruger opdaterer sine oplysninger, kan det ikke ses, før listen nulstilles via en genstart, siden der ikke bliver lavet et nyt fetch på allerede indekserede brugere.

```
let userString;
const uIndex = currentRoom.members.findIndex(u => u.id === messages[i].sender_id);

if (uIndex !== -1) {
  userString = currentRoom.members[uIndex].display_name;
}
else {
  const ueIndex = knownExternalUsers.findIndex(u => u.ID === messages[i].sender_id);

  if (ueIndex !== -1) {
    userString = knownExternalUsers[ueIndex].DisplayName;
  }
  else {
    const callback = await GetUnknownUserInfo(messages[i].sender_id);
    userString = callback.payload.display_name;
  }
}
```

Når beskederne er på plads, bygges medlemslisten. Det er lidt på samme måde som beskederne - information om rummets medlemmer bruges til at bygge en kolonne med elementer, der kan klikkes på. Både venstre- og højreklik bindes til at åbne en minibiografi, som indeholder brugerens offentlige data som synlige navn, brugernavn og profilbillede (dog findes de ikke i denne udgave, så der er i stedet en grå placeholder-cirkel). Hvis man er administrator, kan man også sparke dem ud af et rum. Planen var også at man kunne gøre et medlem til en moderator af et chatrum her, men det har vi ikke nået at implementere. Ligesom med rummenes kontekstmenu, så manipuleres *activeMiniBioID* til at reflektere den relevante brugers ID til et lignende formål.

```
// Tilføjer en ny bruger til brugerlisten
function AddUserToChatroomUserList(user) {
    const userList = document.createElement("div");
    userList.id = "usrid" + user.id;
    userList.classList.add("ChatMembersListing");

    const button = document.createElement("button");
    button.classList.add("ChatMembersListingButton");

    button.onclick = function () { ToggleMiniBio(user.id) };

    button.addEventListener("contextmenu", function(event) {
        event.preventDefault();
        ToggleMiniBio(user.id)
    }, false);

    const userPicture = document.createElement("div");
    userPicture.classList.add("PHCircleSmall");
    button.appendChild(userPicture);

    const userName = document.createElement("span");
    userName.classList.add("LightText");
    userName.innerText = user.display_name;
    button.appendChild(userName);

    userList.appendChild(button);

    chatMembersList.appendChild(userList);
}
```

Forlader brugeren et rum, uanset hvordan, nulstilles en del information. Rummet fjernes fra *cachedRoomIDs*, og hvis det var det åbne chatrum, nulstilles *currentRoom* og *currentRoomID* (dvs man mister medlemslisten og beskederne), chat-elementerne enten fjernes eller gemmes væk, og der bliver lavet et nyt fetch på sine rum for at opdatere listen.

```
// Når man er blevet fjernet fra et chatrum
// På trods af navnet gælder dette også, når man selv forlader et chatrum
function OnKickedFromRoom(roomID) {
    const index = cachedRoomIDs.indexOf(roomID);

    if (index !== -1) {
        // Fjerner rummet fra den lokale liste til Discovery
        cachedRoomIDs.splice(index, 1);

        // Fjerner referencer fra lokal data
        const oldRoomID = currentRoomID;
        currentRoomID = -1;
        currentRoom = null;

        // Hvis rummet er åbent, gemmes elementer væk, og fjerner HTML-info
        if (oldRoomID === roomID) {
            DoShowChatFeatures(false);

            if (chatLogContainer.children.length !== 0) {
                chatLogContainer.removeChild(chatLogContainer.firstChild);
            }

            ClearElementOfChildrenExceptFirst("chatMembersList");
        }

        // Fjerner rummet fra chatlisten/opdaterer chatliste, hvis chatvind
        if (currentTab = "chat") {
            ClearElementOfChildrenExceptFirst("chatList");
            socket.emit("getRoomList");
        }
    }
}
```

Der findes også en del funktioner, der hjælper med at skabe en mere flydende brugeroplevelse. Dette indebærer fx at man kan se nye beskeder i et rum, som de kommer ind, eller se brugere komme til eller forsvinde, som de nu gør, helt uden at skulle efterspørge en manuel opdatering på det hele.

```
// Når det nuværende chatrum får et nyt medlem. Sørger for at vise det med det samme
function OnNewUserJoinedChatroom(userID, userInfo) {
    const newUser = Object.create(null);

    newUser.id = userID;
    newUser.username = userInfo.username;
    newUser.display_name = userInfo.display_name;

    currentRoom.members.push(newUser);

    AddUserToChatroomList(newUser);
}
```

Discovery (Freja)

Her findes der både mulighed for at oprette nye chatrum, men også deltage i dem. Når menuen åbnes, laves der automatisk et fetch på offentlige rum. I øjeblikket gælder det dem allesammen. Den information, der modtages, bruges til at bygge de forskellige elementer i menuens gitter.

```
// Når brugeren modtager listen med offentlige chatrum
function OnReceiveDiscovery(rooms) {
    discoveryRoomsIDs = [];
    UpdateFilterMemberRooms();

    for (let i = 0; i < rooms.length; i++) {
        discoveryRoomsIDs.push(rooms[i].id);

        const box = document.createElement("div");
        box.id = "disid" + rooms[i].id;
        box.classList.add("DiscoveryShowcaseBox");

        const name = document.createElement("span");
        name.innerText = String(rooms[i].name);
        box.appendChild(name);

        const button = document.createElement("button");
        button.innerText = "Deltag";
        button.onclick = function() { TryJoinPublicChatroom(rooms[i].id) };
        box.appendChild(button);

        if (cachedRoomIDs.includes(rooms[i].id)) {
            button.disabled = true;

            if (discoveryFilterMember) {
                box.classList.add("Hidden");
            }
        }

        discoveryGrid.appendChild(box);
    }
}
```

Hvis man filtrerer rum, man allerede er medlem af, gemmes de væk ved at blive tildelt "Hidden"-klassen. Trykker brugeren på "Deltag"-knappen, sendes en efterspørgsel om at blive medlem af det tilknyttede rum. Ved success tilføjes rummet til *cachedRoomIDs* til senere indeksering, og der skiftes til chat-menuen med rummet åbent.

```
// Åbner et nyt chatrum
function RedirectToChatroom(roomID) {
    cachedRoomIDs.push(roomID);
    ChangeActiveWindow("chat");
    socket.emit("tryEnterRoom", roomID);
}
```

Når det kommer til at oprette nye chatrum, gøres begge dele gennem den samme funktion.

```
// Forsøger at oprette et nyt chatrum. Hvis det lykkedes, bliver brugeren automatisk medlem, og det åbnes
function TryCreateChatroom() {
  const nameField = document.getElementById("createChatroomName");
  const passwordField = document.getElementById("createChatroomPassword");
  const privateCheck = document.getElementById("createChatroomPrivateToggle");

  const roomName = nameField.value;
  let roomPassword = "";
  let roomIsPrivate = false;

  if (privateCheck.checked) {
    roomIsPrivate = true;
    roomPassword = passwordField.value;
  }

  if (roomIsPrivate) {
    if (roomPassword !== "") {
      socket.emit("tryCreateChatroom", thisUser.ID, roomName, roomPassword, roomIsPrivate);
    }
    else {
      alert("Kan ikke oprette privat rum uden kode");
    }
  }
  else {
    socket.emit("tryCreateChatroom", thisUser.ID, roomName, "", roomIsPrivate);
  }
}
```

Værdierne i de forskellige input-felter tages ud og sendes med i en efterspørgsel til serveren.

Offentlige rum kan ikke have en kode, og private rum skal have en kode, og man får ikke lov til at sende ufuldstændig data.

Indstillinger (Freja)

Menuen til indstillinger er relativt simpel i forhold til de andre. Den består primært af at sende efterspørgsler til serveren for enten at ændre på brugerdata, logge ud, eller helt slette brugeren.

Ændring af brugerens synlige navn er ikke så spændende. Ændring af brugernavn eller adgangskode foregår gennem den samme funktion, og siden at det er en PATCH-efterspørgsel, hvor tomme felter ignoreres, kan det fint lade sig gøre. Efter at disse oplysninger er blevet ændret, bliver brugeren ført tilbage til login-siden, socket bliver genforbundet, og brugeren skal logge ind igen med sine nye oplysninger.

```
async function TryChangeUserInfo(type) {
  if (!confirm("Er du sikker på, at du vil ændre din information? Ved ær
    return;
  }

  let newUN = "";
  let curPW = "";
  let newPW = "";

  if (type === "username") {
    const unInput = document.getElementById("usernameInput");
    newUN = unInput.value;
  }
  else if (type === "password") {
    const cpwInput = document.getElementById("currentPasswordInput");
    curPW = cpwInput.value;

    const npwInput = document.getElementById("newPasswordInput");
    newPW = npwInput.value;
  }

  const requestOptions = {
    method: "PATCH",
    headers: {
      "Content-type": "application/json"
    },
    body: JSON.stringify({
      username: newUN,
      current_password: curPW,
      new_password: newPW
    })
  };
};
```

Ændring af profilbillede tillader upload af en billedfil, men den kan pt ikke tilgås pga tidsmangel. Når man sletter sin bruger, bliver brugeren ikke 100% slettet. I stedet er det meningen, at den i stedet bliver utilgængelig, og personlig data fjernes. Dette gøres for at holde på beskedhistorik, men vi vil også gerne finde en løsning på at overholde eksempelvis GDPR og så også fjerne beskeder, hvis det ønskes, uden at ødelægge noget internt. Pt er en slettet bruger stadig tilgængelig, og kunne have været undgået med lidt bedre planlægning.

Andet (Freja)

Ideelt ville vi gemme/cache flere ting for at spare flere API-kald, men det har været svært at forudse med mangel på erfaring med dette slags projekt. Som en mindre detalje kunne vi have sparet en smule på forskellige socket events ved at lave flere af dem om fra *emit* til *emitWithAck*, da de fleste har tilhørende feedback.